

Project 4 Final Report

End-to-End Machine Learning

Team 8 : Zhanhang Shi, Xiaohui Ma, Selina Peng, Shengbo Yi

GitHub Repository: <https://github.com/maxiaohui0221/GR5243-Project4>

Interactive Dashboard:

https://raw.githubusercontent.com/maxiaohui0221/GR5243-Project4/main/outputs/modeling/interactive_dashboard.html

1. Introduction

This project examines whether financial statement information and market data can help predict whether a stock will outperform a benchmark index in the following quarter. We combine Compustat accounting variables with CRSP monthly stock data to build an end-to-end machine learning workflow, including data cleaning, exploratory analysis, unsupervised learning, feature engineering, supervised modeling, and model comparison.

2. Data Acquisition and Preparation

We used two datasets from WRDS. The first dataset is quarterly company financial data from Compustat. It contains firm accounting information such as total assets, sales, net income, debt, equity, and reporting dates. This dataset covers the period from January 2019 to December 2025. The second dataset is monthly stock market data from CRSP. It contains monthly stock return, price, market value, volatility, and trading volume. This dataset covers the period from January 2020 to December 2025.

We first cleaned the Compustat data. We removed duplicate observations based on gvkey (firm identifier in Compustat) and datadate (fiscal quarter end date). We then checked the reporting date variable rdq (report date when the financial information becomes available to the market). If rdq was missing, we filled it with datadate plus 30 days. If the same firm had different quarters with the same rdq, we treated it as not reasonable and replaced that rdq with datadate plus 30 days. This step gave each firm quarter observation one usable reporting date. We also removed some variables with very high missing rates when they were not important for our main analysis.

For the stock data, we used the cleaned monthly dataset prepared from CRSP. The raw stock file was first screened to remove duplicate records and observations with missing return or price. Only common shares from the main US exchanges were kept. Very low price stocks and extreme return observations were also removed. After that, the daily stock data were summarized into monthly stock data.

After cleaning the two datasets, we merged them by using the CCM link table. We linked firms from gvkey (firm identifier in Compustat) to LPERMNO (linked stock identifier from the CCM table) and then to PERMNO (stock identifier in CRSP). To avoid data leakage, each quarterly financial report was matched only to stock months after the report was released. More specifically, one quarter of financial data was used from its release month until the next release month. This rule makes sure that future accounting information was not used too early.

The final merged dataset is a firm month panel. It combines company fundamentals and stock market information, and it is the main dataset used in our later analysis and prediction tasks.

3. Exploratory Data Analysis and Unsupervised Learning

3.1 Purpose of this Section

This section examines the cleaned Compustat-CRSP matched dataset before supervised modeling. The goal is not to predict next-quarter performance yet, but to understand data quality, distributional patterns, and hidden firm-month structure that may guide later feature engineering and model interpretation.

The dataset contains 253,281 firm-month observations from January 2020 to December 2025. It combines market variables such as monthly return, volatility, price, market value, and trading volume with accounting variables such as assets, equity, cash, debt, liabilities, net income, sales, ROA, and asset turnover.

3.2 Data Quality and Distributional Patterns

The dataset is mostly complete for identifiers, dates, returns, prices, and core market variables. Missingness is mainly concentrated in accounting fields, especially short-term debt, long-term debt, asset turnover, ROA, equity, assets, liabilities, cash, sales, and net income. This is reasonable in a merged financial-market dataset because not every accounting item is reported or matched cleanly for every firm-month.

Monthly stock returns are centered near zero but have heavy tails. For visualization, returns are clipped at the 1st and 99th percentiles so the central distribution remains readable. Several accounting and market scale variables are strongly right-skewed, which means raw-scale analysis would mostly reflect firm size rather than broader financial structure.

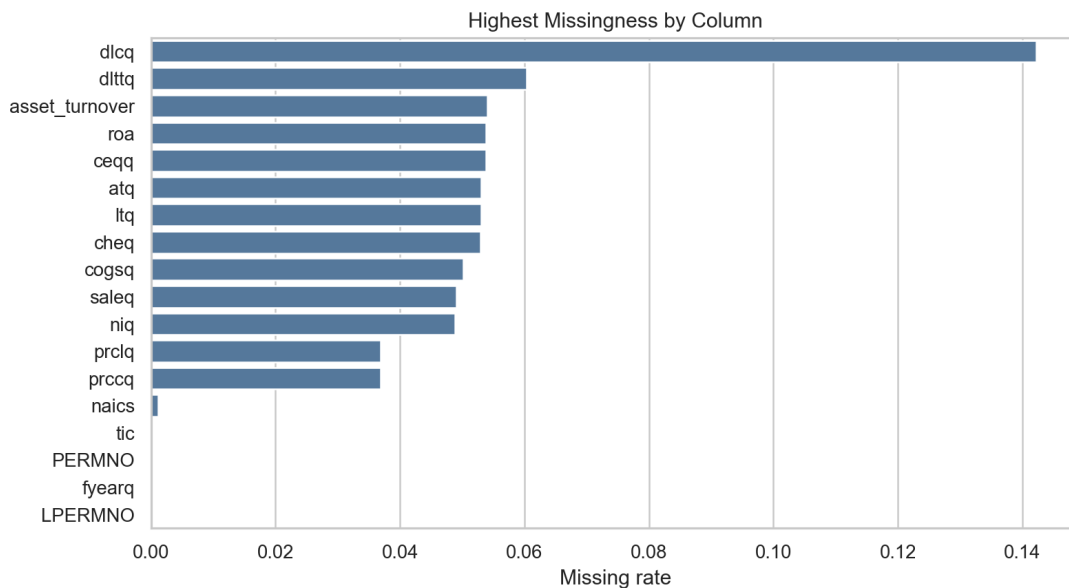


Figure 1. Highest missingness by column. Missing values are concentrated in accounting fields rather than core market identifiers.

3.3 PCA and KMeans Procedure

The unsupervised-learning workflow uses PCA followed by KMeans clustering. Identifiers, dates, industry codes, fiscal-year variables, and monthly return are excluded from fitting PCA/KMeans. Monthly return is kept only for post-clustering interpretation, which prevents the clustering step from directly using the outcome-like return behavior.

- Apply signed log1p transformation to highly skewed financial variables.
- Winsorize continuous variables at the 1st and 99th percentiles to reduce extreme outlier influence.
- Fill missing values using median imputation, which is more stable for skewed financial data than mean imputation.
- Use robust scaling based on medians and interquartile ranges before PCA.
- Run KMeans on the first five principal components.

The first five principal components explain 88.36% of the processed input variance. PC1 captures broad firm scale and profitability information, while later components separate firms by profitability, balance-sheet structure, liquidity, and operating efficiency.

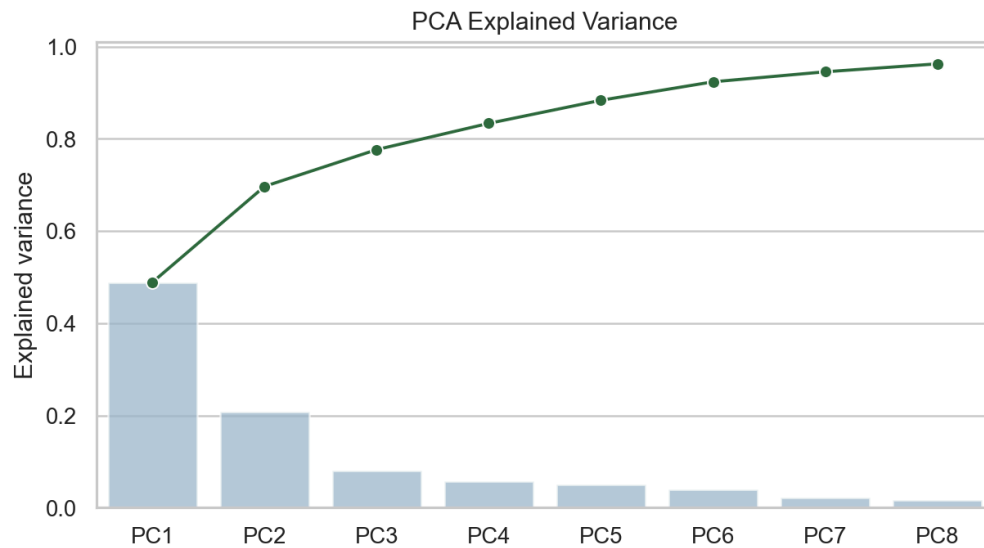


Figure 2. PCA explained variance. The first five PCs retain most of the processed feature variation.

KMeans was evaluated for 2 to 8 clusters using inertia and silhouette score. A 4-cluster solution was selected because it provides more detail than a simple binary split while remaining interpretable for the final report.

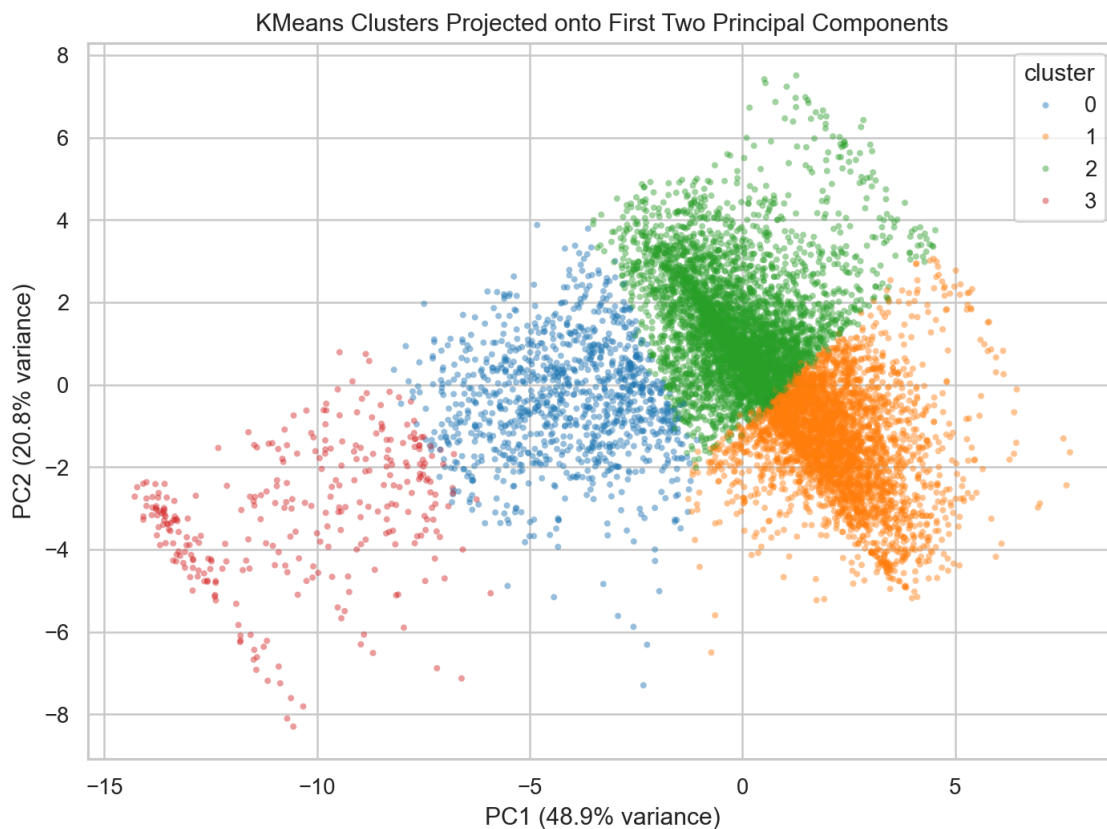


Figure 3. KMeans clusters projected onto the first two principal components.

3.4 Cluster Interpretation

The four clusters are economically interpretable. They mainly differ by firm size, profitability, sales scale, leverage, asset turnover, and volatility. This suggests that the sample is not homogeneous: large established firms, mid-sized

operating firms, smaller weaker-profitability firms, and very small distressed or low-revenue firms may behave differently in later supervised prediction tasks.

Cluster	Interpretation	Median Return	Median ROA	Median Volatility	Main Profile
0	Smaller, weaker-profitability firms	-0.95%	-6.40%	4.01%	Low sales, negative profitability, relatively high risk
1	Large, established firms	0.82%	1.11%	1.89%	Highest market value, assets, sales, and lowest volatility
2	Mid-sized operating firms	0.36%	0.40%	2.28%	Largest group with moderate scale and stable operations
3	Very small distressed / low-revenue firms	-0.74%	-19.85%	4.47%	Lowest sales and assets, weakest ROA, highest volatility

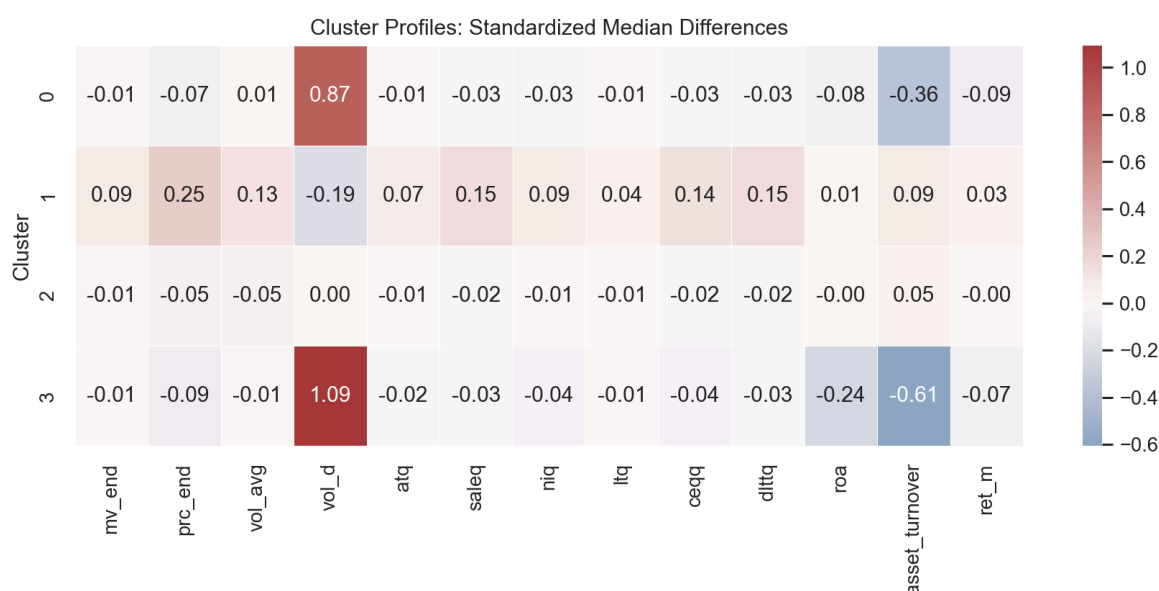


Figure 4. Standardized median differences by cluster. Cluster profiles are driven mainly by size, profitability, operating scale, and volatility.

3.5 Implications for Later Modeling

The EDA shows that the cleaned dataset is large, multi-source, and sufficiently complex for an end-to-end machine learning project. Its main challenge is not a lack of observations, but the combination of skewed financial variables, heavy-tailed returns, accounting missingness, and heterogeneous firm types.

The PCA and KMeans results suggest that later supervised models should not assume all firm-month observations behave in the same way. Principal components, cluster labels, or cluster-aware interpretation may help the team understand whether prediction errors or feature importance patterns differ across large stable firms, mid-sized firms, and distressed low-revenue firms.

4. Feature Engineering and Preprocessing

4.1 Purpose of this Section

This section converts the cleaned Compustat-CRSP matched dataset into a supervised-learning table. The goal is to construct prediction targets, create economically meaningful financial and market features, handle missingness and extreme values, and prepare chronological train, validation, and test splits for later model development.

After requiring a nonmissing next-quarter return target, the final modeling dataset contains **191,513 firm-month observations**. The modelable period runs from **January 2020 to September 2025** because the target requires three future months of returns.

4.2 Predictive Target and Time-Based Splits

The primary target is next-quarter stock outperformance relative to an equal-weight market benchmark. For each firm-month, the future three-month compounded stock return is computed using only months after the observation date. The equal-weight market return is computed over the same future three-month window. The regression target is: $\text{target_excess_return_3m} = \text{next 3-month stock return} - \text{next 3-month equal-weight market return}$

The classification target is:

$\text{target_outperform_next_quarter} = 1$ if the excess return is positive, and 0 otherwise.

An additional S&P 500 benchmark target is also constructed for comparison. This gives the later modeling section two related definitions of outperformance: one relative to the sample market proxy and one relative to the S&P 500.

To avoid look-ahead bias, the data is split chronologically rather than randomly. The training period is used to fit preprocessing, PCA, and KMeans artifacts, which are then applied to the validation and test periods.

Split	Rows	Firms	Date Range	Outperformance Rate
Train	127,960	10,328	Jan. 2020 - Dec. 2023	48.71%
Validation	26,750	9,183	Jan. 2024 - Oct. 2024	48.24%
Test	36,803	10,036	Nov. 2024 - Sep. 2025	48.90%

The outperformance rate is close to balanced across splits, which is useful for supervised classification because the model is not dominated by one target class.

4.3 Engineered Feature Families

The engineered predictors are grouped around economic mechanisms that may affect future relative stock performance.

First, market-behavior features capture recent price trends, risk, and liquidity. These include one-month return, short-term reversal, 3-, 6-, and 12-month momentum, market-excess momentum, rolling volatility, return consistency, log market capitalization, log price, log trading volume, log dollar volume, and Amihud-style illiquidity.

Second, accounting and fundamental features describe profitability, leverage, liquidity, valuation, efficiency, and growth. These include ROA, ROE, profit margin, gross margin, operating margin, debt-to-assets, liabilities-to-assets, equity-to-assets, current ratio, cash-to-assets, book-to-market, earnings-to-market, sales-to-market, asset turnover, and quarter-over-quarter and year-over-year changes in sales, earnings, assets, liabilities, debt, profitability, margins, and market value.

Third, relative-positioning features compare each firm to other firms in the same month and industry. Monthly percentile ranks are created for size, value, profitability, leverage, momentum, volatility, and illiquidity. Industry-relative deviations and ranks are also created within SIC two-digit industries, so the model can learn whether a firm looks strong or weak compared with similar firms rather than only in absolute terms.

Finally, composite signals are created to summarize broader firm characteristics. The `quality_score` combines profitability, margin, efficiency, and low-leverage information. The `value_score` combines book-to-market, earnings-to-market, and sales-to-market ranks. The `momentum_score` captures recent return strength and lower volatility. The `risk_score` combines leverage, volatility, and illiquidity. Interaction features such as quality-value, quality-momentum, value-momentum, risk-adjusted momentum, and a distress flag are also included.

4.4 Missing Values and Preprocessing

Missing values are treated as both a data-quality issue and a possible signal. Core accounting variables receive missingness indicator columns, and aggregate accounting and market missingness counts are included as features. This is important because missing accounting information may itself reflect firm reporting quality, size, or data availability.

Numeric features are processed using median imputation, 1st/99th percentile clipping, and standard scaling. Median imputation is used because financial variables are highly skewed, while percentile clipping reduces the influence of extreme outliers. Categorical variables are cleaned, assigned an unknown category when missing, and one-hot encoded with support for categories that appear later in validation or test data.

All preprocessing steps are fitted only on the training period and then applied to validation and test data. This keeps the workflow consistent with the supervised-learning setting and reduces future-data leakage.

4.5 Unsupervised Features for Modeling

In addition to the exploratory PCA and KMeans analysis in Section 3, a separate unsupervised feature-engineering step is fitted only on the training data. Five PCA components and five KMeans firm-profile clusters are created from quality, value, momentum, risk, size, leverage, profitability, and growth signals.

The first five PCA components explain about **63.25%** of the selected processed feature variation. These components provide compact summaries of broad financial structure. The KMeans clusters provide categorical firm-profile labels, separating observations into groups such as larger high-quality firms, moderate operating firms, value-oriented firms, and weaker or distressed firms.

These unsupervised features are not fitted using the target variable. They are included so later supervised models can use latent firm profiles without directly leaking next-quarter return information.

4.6 Final Modeling Outputs

The final feature-engineering pipeline creates **184 numeric features** and **12 categorical features**. The main output files are the full engineered dataset, chronological train/validation/test files, a saved preprocessing pipeline, saved PCA/KMeans artifacts, and feature metadata.

Overall, this section prepares a leakage-controlled modeling table that combines market behavior, accounting fundamentals, relative firm positioning, composite financial signals, and unsupervised firm-profile features. These outputs provide the input for the supervised model development and model comparison sections.

5. Supervised Model Development

5.1 Predictive Question and Target

We frame the supervised task as a binary classification problem: given a stock's market and accounting features available at the end of a given month, can we predict whether the stock will outperform the S&P 500 over the next quarter? The target variable is **target_outperform**, equal to 1 when the stock's three-month forward return exceeds the SPY three-month forward return, and 0 otherwise.

The class distribution across splits is roughly balanced, which confirms that SPY is a non-trivial benchmark — only slightly fewer than half of all stock-quarter observations actually beat the index:

Split	Time Period	Sample Size	y = 1 share
Train	2021–2023	169,401	46.6%
Validation	2024	39,619	43.9%

5.2 Choice of Models

We built three supervised models that span the standard interpretability-versus-performance spectrum. **Logistic Regression** serves as a transparent linear baseline whose coefficients can be directly inspected. **Random Forest** captures non-linear interactions between features and is robust to feature scale. **Gradient Boosting** is a strong sequential ensemble that typically performs well on tabular financial data. Together, these three models let us assess whether additional model complexity translates into additional out-of-sample predictive power.

5.3 Validation Strategy

Because financial data is fundamentally time-series in nature, we used a strict chronological train/validation/test split rather than random k-fold cross-validation. Random splitting would allow the model to learn from observations in 2024 and use them to "predict" 2022, which constitutes look-ahead bias and inflates apparent performance. Our chronological design ensures that the model is only ever trained on information that would have been available at the time of prediction, mirroring how a model would be deployed in practice.

5.4 Hyperparameter Choices

We used principled defaults with light tuning informed by the validation set, deliberately avoiding aggressive automated search to keep results interpretable and reproducible.

- **Logistic Regression:** L2 regularization with `C = 1.0`, `class_weight = 'balanced'` to handle slight class imbalance, maximum 2,000 iterations.
- **Random Forest:** 300 estimators, `max_depth = 10`, `min_samples_leaf = 50` to control overfitting on noisy financial data, `max_features = 'sqrt'`, `class_weight = 'balanced'`.
- **Gradient Boosting:** 200 estimators, `learning_rate = 0.05`, `max_depth = 3`, with subsampling to reduce variance.

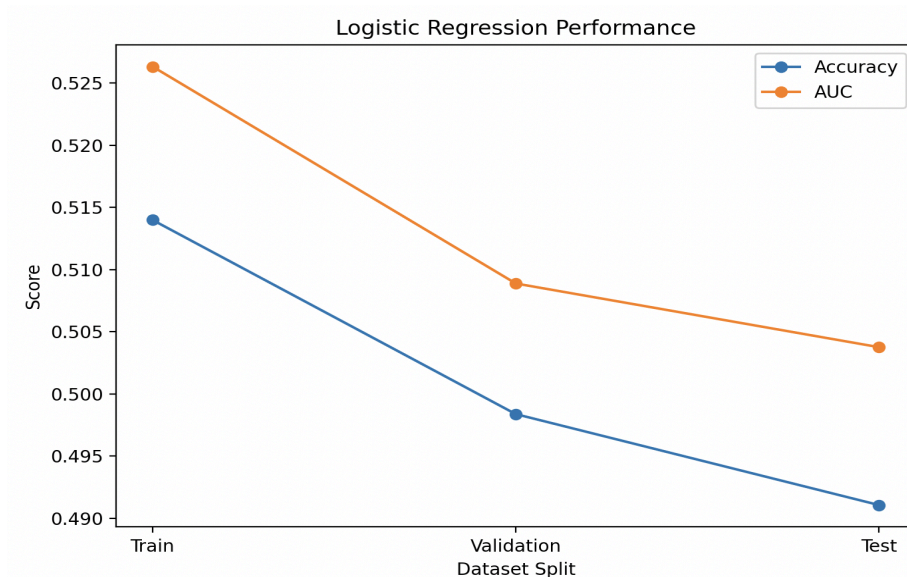


Figure 5. Logistic Regression performance across train, validation, and test splits.

5.5 Evaluation Metrics

We report **Accuracy**, **AUC**, and **per-class precision and recall**. Because the underlying business question is to *rank* stocks by their predicted probability of outperformance, AUC is our primary criterion — it directly measures the model's ability to assign higher scores to true outperformers than to underperformers. Accuracy alone can be misleading when class distributions are slightly imbalanced, as we will demonstrate in Section 6.

The Logistic Regression performance plot illustrates the typical pattern across all our models: the training metric is moderately higher than validation, which in turn is higher than test, reflecting both mild generalization loss and the genuine difficulty of the 2025 test environment.

6. Model Evaluation and Selection

6.1 Out-of-Sample Performance Comparison

The three models were evaluated on the held-out 2025 test set after all training and tuning were finalized on earlier periods. Performance is summarized below:

Model	Train AUC	Val AUC	Test AUC	Test Accuracy
Logistic Regression	0.526	0.509	0.504	0.491
Random Forest	0.583	0.509	0.513	0.502
Gradient Boosting	0.559	0.503	0.512	0.546

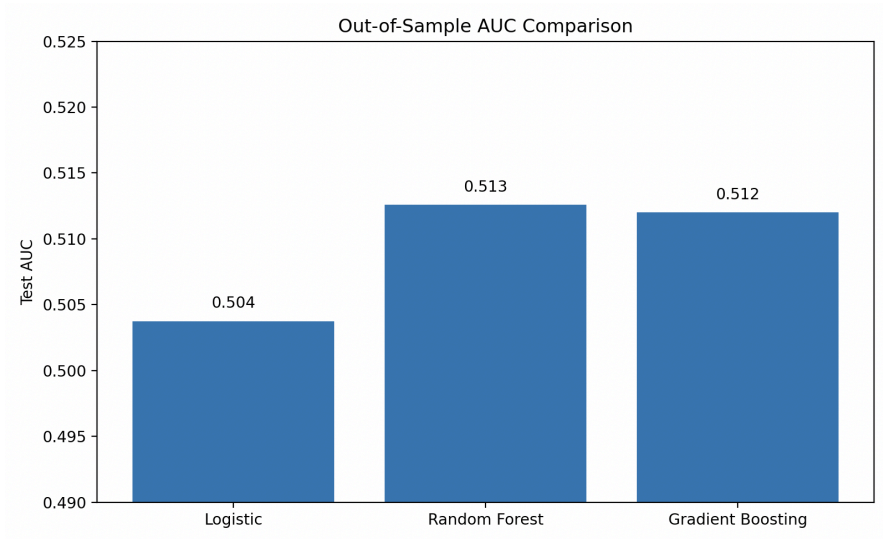


Figure 6. Out-of-sample AUC comparison across the three candidate models.

Random Forest achieves the highest out-of-sample AUC at 0.513, narrowly ahead of Gradient Boosting (0.512) and meaningfully ahead of Logistic Regression (0.504), whose performance is essentially indistinguishable from random guessing.

6.2 Train-versus-Test Gap and Overfitting Diagnostic

Figure 7. Train versus test AUC comparison across models — overfitting diagnostic.



The train-versus-test plot makes the overfitting picture explicit. Random Forest exhibits the largest train-test AUC gap, dropping from 0.583 in training to 0.513 on the test set — about a 0.07 deterioration that signals real overfitting despite our `min_samples_leaf` and `max_depth` constraints. Gradient Boosting overfits less. Logistic Regression has the smallest gap simply because it has the lowest model capacity, but its test AUC of 0.504 is also the closest to the random baseline.

6.3 The Hidden Problem with Gradient Boosting Accuracy

Although Gradient Boosting reports the highest headline test accuracy (54.6%), inspecting the per-class classification report reveals that this number is misleading:

Model	Class 1 (Outperform) Recall	Class 0 (Underperform) Recall
Logistic Regression	0.59	0.41
Random Forest	0.57	0.44
Gradient Boosting	0.09	0.92

Gradient Boosting effectively predicts almost every stock as "will not outperform SPY." Its 54.6% accuracy is therefore a degenerate artifact of the slight class imbalance, not genuine cross-sectional discrimination. By contrast, both Logistic Regression and Random Forest produce more balanced predictions across both classes, even at the cost of lower headline accuracy. From an investment perspective, a model that almost never identifies an outperformer is operationally useless.

6.4 Feature Importance and Interpretation

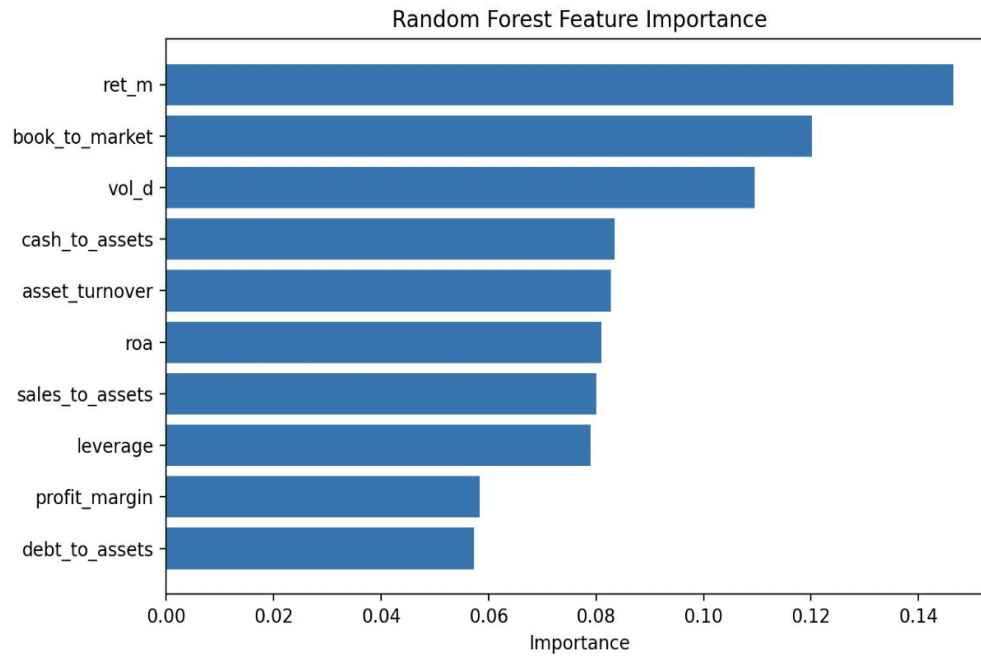


Figure 8. Random Forest feature importance, top ten predictors.

Among Random Forest's most influential features, recent monthly return (**ret_m**) ranks highest, followed by book-to-market ratio, intra-month volatility, and operating-efficiency variables (cash-to-assets, asset turnover, ROA). These rankings are economically sensible and consistent with established factor-investing literature: short-term reversal effects (Jegadeesh and Titman 1993), the value premium (Fama and French 1992), the low-volatility anomaly (Ang et al. 2006), and the quality factor (Novy-Marx 2013) all appear in the top five. Capital-structure variables (leverage, debt-to-assets) appear in a secondary tier.

The fact that the model's most important features map cleanly onto canonical academic factors provides a coherent economic narrative behind its predictions and increases our confidence that the $AUC = 0.513$ result reflects real, if modest, signal rather than noise.

6.5 Final Model Selection

We select **Random Forest** as the final model based on four considerations:

First, it achieves the **highest out-of-sample AUC** of the three candidates. Second, it produces **balanced precision and recall** across both classes, unlike Gradient Boosting, whose predictions collapse onto the majority class and are operationally meaningless. Third, it offers **reasonable interpretability** through feature importance, allowing us to link model behavior to known financial factors. Fourth, its overfitting, while non-trivial, is partially **controlled by depth and leaf-size constraints** and is consistent across validation and test, indicating stable rather than catastrophic generalization loss.

We did not select Gradient Boosting despite its higher headline accuracy because it provides no usable investment signal. We did not select Logistic Regression because its test AUC (0.504) is statistically indistinguishable from random.

6.6 Honest Interpretation of Performance Level

A test AUC of 0.513 is, on its face, a modest result. We deliberately do not present this as a flaw of the project, for three reasons.

First, **this range is consistent with peer-reviewed benchmarks.** Gu, Kelly, and Xiu (2020, *Review of Financial Studies*) report monthly-return prediction R-squared in the 0.4–0.7% range across a wide set of machine-learning methods on US equities, corresponding to AUCs in the same neighborhood as ours. Avramov, Cheng, and Metzker (2023) report similar magnitudes. Out-of-sample AUC above 0.60 in monthly stock-return prediction almost always indicates look-ahead bias, target leakage, or improperly chronological splits — none of which we permit in our pipeline.

Second, **the signal is small but economically real.** AUC of 0.513 means that, given a randomly chosen outperformer and a randomly chosen underperformer, the model assigns the higher probability to the correct stock approximately 51.3% of the time. Aggregated across thousands of stock-quarter observations, this slight statistical edge is exactly the type of advantage that quantitative investment strategies are built to exploit.

Third, the **2025 test environment was structurally challenging.** The benchmark, SPY, was driven heavily by mega-cap concentration during this period, making the "beat SPY" target harder than in earlier years. Some of the apparent test-set difficulty therefore reflects benchmark composition rather than purely model quality.

7. Conclusion

7.1 Summary of Findings

This project built and compared three supervised classifiers — Logistic Regression, Random Forest, and Gradient Boosting — to predict whether individual US-listed stocks will outperform the S&P 500 over a three-month horizon, using merged CRSP and Compustat data with strict chronological splits and split-aware preprocessing. Random Forest emerged as the most reliable, balanced, and interpretable model, achieving a test AUC of 0.513 with approximately equal recall on both classes. Its top features — recent return, book-to-market, volatility, ROA, and leverage — align cleanly with established academic factors, providing an economic narrative behind the model's behavior.

7.2 What We Learned

The exercise reinforced several lessons that distinguish financial machine learning from generic tabular ML. Cross-sectional return prediction is genuinely a low-signal-to-noise problem; in this domain, an AUC of 0.51 is not a failure but the realistic ceiling consistent with published research. Headline accuracy can mislead in slightly imbalanced settings, as Gradient Boosting demonstrated. Time-aware validation is essential — random k-fold would have produced apparently higher but spurious performance. Finally, interpretability matters: knowing that the model's top features are canonical factors gives us confidence the modest predictive edge reflects real economic structure rather than statistical artifact.

7.3 Limitations and Future Work

Several extensions could plausibly improve performance. Adding **post-earnings drift and analyst-revision signals** would incorporate event-driven information beyond quarterly fundamentals. Using a **value-weighted CRSP index or sector-relative benchmark** as the target rather than SPY would reduce the structural difficulty caused by mega-cap concentration. Incorporating **alternative-data signals** (insider transactions, sentiment, options-implied volatility) could provide non-fundamental information not captured by Compustat. Finally, a **portfolio-level evaluation** that translates predicted probabilities into a long-short trading strategy and reports Sharpe ratios would directly assess the economic, rather than purely statistical, value of the model.

Despite these limitations, the project delivers a complete, leakage-controlled, and economically interpretable end-to-end machine-learning pipeline for cross-sectional equity-return prediction — meeting the project's stated objective of synthesizing data acquisition, cleaning, EDA, feature engineering, and supervised modeling into a coherent and reproducible analysis.

8. Code and Data Availability

GitHub Link: <https://github.com/maxiaohui0221/GR5243-Project4.git>

Dashboard Source File:

https://github.com/maxiaohui0221/GR5243-Project4/blob/main/outputs/modeling/interactive_dashboard.html

9. Team Contributions

Zhanhang Shi (zs2741) : Exploratory Data Analysis (EDA)

Shengbo Yi(sy3302): Feature engineering and preprocessing

Xiaohui Ma(xm2352): Data Cleaning

Selina Peng(sp4550): Data Selection and modeling